

WEEK 4 MANDATORY HANDS ON --- Microservices with Spring Boot 3 and Spring Cloud

Creating Microservices for account and loan

In this hands on exercises, we will create two microservices for a bank

Directory Structure Setup

Before generating the projects, set up your workspace directory on your local machine.

1. Navigate to your D: Drive.
2. Create a folder named after your Employee ID (e.g., D:\123456).
3. Inside your Employee ID folder, create a new folder named microservices (e.g., D:\123456\microservices). This will act as your workspace.

2. Creating the Account Microservice

Step A: Generate the Project via Spring Initializr

1. Open your browser and navigate to start.spring.io.
2. Configure the metadata with the following values:
 - Project: Maven Project
 - Language: Java
 - Spring Boot: (Select the latest stable pre-selected version)
 - Group: com.cognizant
 - Artifact: account

- Packaging: Jar
 - Java: (Select your installed JDK version, e.g., 17 or 21)
3. Under Dependencies, search for and add:
 - Spring Web (Web > Spring Web)
 - Spring Boot DevTools (Developer Tools > Spring Boot DevTools)
 4. Click Generate to download the account.zip file.
 5. Extract the zip file and place the account project folder directly into your microservices directory (e.g., D:\123456\microservices\account).

Step B: Build the Project

1. Open your Command Prompt (CMD).
2. Navigate to your account project directory:

DOS

```
cd D:\123456\microservices\account
```

3. Run the Maven build command:

DOS

```
mvn clean package
```

(Verify that you see a "BUILD SUCCESS" message).

Step C: Implement the Account Controller

1. Open Eclipse IDE.
2. Go to File > Import > Existing Maven Projects, browse to your D:\123456\microservices\account folder, and click Finish.
3. Create a new class named AccountController.java in the package com.cognizant.account.

4. Replace its contents with the following code:

Java

```
package com.cognizant.account;

import java.util.HashMap;
import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class AccountController {

    @GetMapping("/accounts/{number}")
    public Map<String, Object> getAccountDetails(@PathVariable
String number) {
        Map<String, Object> response = new HashMap<>();
        response.put("number", number);
        response.put("type", "savings");
        response.put("balance", 234343);
        return response;
    }
}
```

Step D: Run and Test

1. Right-click on AccountApplication.java (found under src/main/java/com/cognizant/account) and select Run As > Spring Boot App (or Java Application).
2. Open your browser and test the endpoint:

Plaintext

<http://localhost:8080/accounts/00987987973432>

3. Expected JSON Response:

JSON

```
{  
  "number": "00987987973432",  
  "type": "savings",  
  "balance": 234343  
}
```

3. Creating the Loan Microservice

Step A: Generate the Project

1. Go back to start.spring.io.
2. Keep the configuration the same as the Account microservice, but change the Artifact name:
 - Group: com.cognizant
 - Artifact: loan
3. Ensure Spring Web and Spring Boot DevTools dependencies are selected.
4. Click Generate to download the loan.zip file.
5. Extract and move the loan folder to your microservices workspace directory (e.g., D:\123456\microservices\loan).

Step B: Build and Import

1. Open your command prompt, navigate to the loan directory, and build it:

DOS

```
cd D:\123456\microservices\loan
```

```
mvn clean package
```

2. Import the project into Eclipse as an Existing Maven Project.

Step C: Configure Port to Prevent Address Bind Conflict

Because the Account Microservice is already running on port 8080, running the Loan Microservice on the same port will cause a `java.net.BindException: Address already in use` error. We must assign a different port to this service.

1. In Eclipse, open the loan project.
2. Locate and open `src/main/resources/application.properties` (or `application.yml`).
3. Add the following line to configure the port:

Properties

```
server.port=8081
```

4. Save the file.

Step D: Implement the Loan Controller

1. Create a new class named `LoanController.java` in the package `com.cognizant.loan`.
2. Add the following controller implementation:

Java

```
package com.cognizant.loan;
```

```
import java.util.HashMap;
import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;
```

```
@RestController
```

```
public class LoanController {
```

```
    @GetMapping("/loans/{number}")
```

```
    public Map<String, Object> getLoanDetails(@PathVariable String
number) {
```

```
        Map<String, Object> response = new HashMap<>();
```

```
        response.put("number", number);
```

```
        response.put("type", "car");
```

```
        response.put("loan", 400000);
```

```
        response.put("emi", 3258);
```

```
        response.put("tenure", 18);
```

```
        return response;
```

```
    }
```

```
}
```

Step E: Run and Test

1. Ensure your Account Microservice is still running.

2. Right-click on LoanApplication.java inside the loan project and select Run As > Spring Boot App.
3. Open your browser and navigate to:

Plaintext

<http://localhost:8081/loans/H00987987972342>

4. Expected JSON Response:

JSON

```
{  
  "number": "H00987987972342",  
  "type": "car",  
  "loan": 400000,  
  "emi": 3258,  
  "tenure": 18  
}
```

4. Managing Multiple Consoles in Eclipse

When running both microservices simultaneously, Eclipse consolidates output views. To monitor individual application logs:

Create Eureka Discovery Server and register

Microservices

1. Setting up Eureka Discovery Server

The Discovery Server serves as a registry where all microservices register themselves dynamically, eliminating the need to hardcode service URLs.

Step A: Generate the Project

1. Open start.spring.io in your browser.
2. Enter the configuration:
 - **Group:** com.cognizant
 - **Artifact:** eureka-discovery-server
 - **Dependency: Eureka Server** (Spring Cloud Discovery > Eureka Server)
3. Download, extract, and import the project into Eclipse.

Step B: Configure the Discovery Server

1. Open EurekaDiscoveryServerApplication.java and add the @EnableEurekaServer annotation:

Java

```
package com.cognizant.eurekadiscoveryserver;
```

```
import org.springframework.boot.SpringApplication;
```

```
import
```

```
org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
import
```

```
org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
```

```
@SpringBootApplication
```

```
@EnableEurekaServer
```

```
public class EurekaDiscoveryServerApplication {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(EurekaDiscoveryServerApplication.class,  
args);
```



```
}  
}
```

2. Open `src/main/resources/application.properties` and paste the following setup:

Properties

`server.port=8761`

`eureka.client.register-with-eureka=false`

`eureka.client.fetch-registry=false`

`logging.level.com.netflix.eureka=OFF`

`logging.level.com.netflix.discovery=OFF`

3. **Run** the server as a Java/Spring Boot Application.
4. Open your browser and navigate to `http://localhost:8761`. You will see the Eureka Dashboard, and under **Instances currently registered with Eureka**, the list will be empty.

2. Registering Account and Loan Microservices with Eureka

Step A: Update the `pom.xml` Dependencies

To make your existing **Account** and **Loan** microservices act as discovery clients, open their respective `pom.xml` files and add the Eureka Client dependency.

1. Add the **Spring Cloud Starter Netflix Eureka Client** dependency inside the `<dependencies>` block of both `account` and `loan` `pom.xml` files:

XML

`<dependency>`

`<groupId>org.springframework.cloud</groupId>`

`<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>`

</dependency>

2. Ensure your parent <dependencyManagement> section includes the Spring Cloud BOM (Bill of Materials) if it isn't already present:

XML

<dependencyManagement>

<dependencies>

<dependency>

<groupId>org.springframework.cloud</groupId>

<artifactId>spring-cloud-dependencies</artifactId>

<version>2023.0.3</version> <type>pom</type>

<scope>import</scope>

</dependency>

</dependencies>

</dependencyManagement>

Step B: Update Code and Configuration

1. For Account Service

- Open AccountApplication.java and add the @EnableDiscoveryClient annotation.
- Open application.properties and add:

Properties

spring.application.name=account-service

server.port=8080

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

2. For Loan Service

- Open LoanApplication.java and add the @EnableDiscoveryClient annotation.
- Open application.properties and add:

Properties

spring.application.name=loan-service

server.port=8081

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

Step C: Test Registry

1. Stop all current running services in Eclipse.
2. Start **Eureka Discovery Server** first and wait for it to run.
3. Start **Account Microservice** and **Loan Microservice**.
4. Refresh http://localhost:8761. You should now see **ACCOUNT-SERVICE** and **LOAN-SERVICE** listed under registered instances.

3. Creating the Spring Cloud API Gateway

We will now route incoming client requests through a single entry point (the API Gateway) instead of hitting each individual microservice port directly.

Step A: Generate the Gateway Service

1. Generate a new project on Spring Initializr:
 - **Group:** com.cognizant
 - **Artifact:** api-gateway
 - **Dependencies:** **Gateway** (Routing > Gateway), **Eureka Discovery Client** (Spring Cloud Discovery > Eureka Discovery Client)
2. Extract, import, and build the project in Eclipse.

Step B: Configure Routing

1. Open application.properties in your new api-gateway project:

Properties

server.port=9090

spring.application.name=api-gateway

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

Enable routing locator using Eureka discovery service-id naming

spring.cloud.gateway.discovery.locator.enabled=true

spring.cloud.gateway.discovery.locator.lower-case-service-id=true

Step C: Implement Global Logging Filter

Create a global filter to capture and print incoming request paths in the API gateway logs.

1. Create a class LogFilter.java inside com.cognizant.apigateway:

Java

```
package com.cognizant.apigateway;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
```

```
import org.springframework.cloud.gateway.filter.GlobalFilter;
```

```
import org.springframework.stereotype.Component;
```

```
import org.springframework.web.server.ServerWebExchange;
```

```
import reactor.core.publisher.Mono;
```

@Component

```
public class LogFilter implements GlobalFilter {
```

```
    private final Logger logger =  
    LoggerFactory.getLogger(LogFilter.class);
```

@Override

```
    public Mono<Void> filter(ServerWebExchange exchange,  
    GatewayFilterChain chain) {
```

```
        // Pre-filter processing: Log target request path
```

```
        logger.info("Global Log Filter triggered: Request received for URI  
-> {}", exchange.getRequest().getPath());
```

```
        return chain.filter(exchange).then(Mono.fromRunnable(() -> {
```

```
            // Post-filter processing: Code runs after target service  
            responds
```

```
            logger.info("Global Log Filter ended: Response status -> {}",  
            exchange.getResponse().getStatusCode());
```

```
        }));
```

```
    }
```

```
}
```

4. End-to-End Test Execution

1. Start all 4 services in the following order:

- **Eureka Server (8761)**

- **Account Service** (8080)
 - **Loan Service** (8081)
 - **API Gateway** (9090)
2. Access the Microservices using the Gateway port **9090** and lower-case service names:
- **Account:** Navigate to <http://localhost:9090/account-service/accounts/00987987973432>
 - **Loan:** Navigate to <http://localhost:9090/loan-service/loans/H00987987972342>
3. View the console logs inside your **API Gateway** tab in Eclipse. You should see outputs detailing:

Plaintext

INFO com.cognizant.apigateway.LogFilter - Global Log Filter triggered: Request received for URI -> /account-service/accounts/00987987973432

INFO com.cognizant.apigateway.LogFilter - Global Log Filter ended: Response status -> 200 OK